

Assignment01: Git-Based Collaboration for an ML Project

Overview

In teams of 2–3, you will take an existing ML dataset and training code, put it under Git and DVC, and run it like a real team project: branches, experiments, reviewed pull requests, versioned data, CI and a tagged release. The goal is the one from the course: **every result, reproducible by anyone on the team.**

What you will practise

- A clean repo layout, `.gitignore` and pre-commit hooks (Session 1)
- A `dev` → `staging` → `main` workflow with feature and experiment branches and reviewed PRs (Session 2)
- Notebooks that diff and merge cleanly with `nbstripout` and `jupyter` (Session 3)
- Data and model versioning with DVC (Session 4)
- Reproducible experiments: commit + config + data + environment + seed (Session 5)
- CI checks on every PR and a reproducible release (Session 6)

Team rules

- 2–3 students per team. One repository per team, hosted on GitHub (or GitLab), with all members as collaborators and the instructor added as a viewer.
- Every member must author commits, open PRs and review a teammate's PRs. Contribution is checked in the Git history, so do not commit on each other's behalf.
- Nobody pushes directly to `dev`, `staging` or `main`. All changes arrive through pull requests.

What you submit: the repository link, a `REPORT.md` in the repo, and a release tag `model-v1.0` on `main` that a stranger can reproduce.

Branching model

Work flows in one direction: short-lived branches merge into `dev`, `dev` is promoted to `staging`, and `staging` is promoted to `main`. Only `main` represents production.

Branch	Purpose	Created from	Merges into	Protection
<code>main</code>	Production: released, tagged models only	—	—	PR only, 1 approval, all CI checks pass
<code>staging</code>	Release candidate, reproduced and validated before release	<code>main</code>	<code>main</code>	PR only, 1 approval, all CI checks pass
<code>dev</code>	Integration of	<code>main</code>	<code>staging</code>	PR only, 1

Branch	Purpose	Created from	Merges into	Protection
	finished work			approval, CI passes
feat/<name>	Production code: features, pipeline changes	dev	dev	Deleted after merge
data/<name>	Dataset updates tracked with DVC	dev	dev	Deleted after merge
exp/<member>-<idea>	Exploration; may never merge	dev	Nothing directly: cherry-pick the winner into a feat/ branch	Keep short-lived, rebase on dev often
fix/<name>	Urgent fix to production	main	main, then back into dev	Deleted after merge

The branch model has three permanent branches and four kinds of short-lived ones. Work always moves in one direction, from dev to staging to main, and only through reviewed pull requests with passing CI.

- **dev (integration):** where finished work comes together. Every feat/ and data/ branch starts from dev and merges back into it through a PR that a teammate has reviewed.
- **staging (release candidate):** when dev is ready, open a release PR into staging. A teammate who did not train the model clones the repo fresh, runs `dvc pull` and `dvc repro`, and confirms the metrics match exactly.
- **main (production):** only reproduced, approved code from staging reaches main. Every merge into main is tagged, for example `model-v1.0`.
- **feat/<name>:** production code changes, such as new features, pipeline steps or refactors. Keep these small and short-lived, and delete them after merging.
- **data/<name>:** dataset changes tracked with DVC. Always run `dvc push` before `git push`, or your teammates will get broken data pointers.
- **exp/<member>-<idea>:** your personal space for trying ideas. These branches are never merged directly. When an experiment wins, cherry-pick or re-apply the change onto a new feat/ branch and open a PR from there. It's fine to abandon experiment branches.
- **fix/<name>:** urgent fixes to production. Branch from main, merge back into main with a patch tag such as `model-v1.0.1`, then merge main back into dev so the fix isn't lost in the next release.

Use [Conventional Commits](#) for messages, for example feat: add scaling step, data: remove duplicate rows, exp: try max_depth=8. As a team, decide once whether PRs into dev are squash-merged or rebase-merged, and write that decision in `CONTRIBUTING.md`.

Step-by-step tasks

Complete the nine phases in order. Each phase ends with a checkpoint you must be able to show in the repository.

Phase 1 · Team and repository setup

1. Form a team of 2–3 and pick a team name. Choose a dataset and starter code from the list at the end of this document.
2. One member creates an empty repository named `<team>-ml-collab` and adds the teammates as collaborators with write access, plus the instructor.
3. Every member clones it and sets their name and email: `git config user.name` and `git config user.email`.
4. Assign roles for the project (each person owns one, but everyone codes and reviews):
 - o **Data owner:** DVC, data checks, dataset updates
 - o **Model owner:** training pipeline, configs, experiments
 - o **Platform owner:** CI, pre-commit, environment, releases (in a team of 2, split this role)

Checkpoint: all members can push a branch to the repository.

Phase 2 · Scaffold the project and import the initial code

1. Create the standard layout (you may start from [Cookiecutter Data Science](#)):

```
.
├── configs/           # params.yaml lives here or at the root
├── data/              # ignored by Git, tracked by DVC
├── models/            # ignored by Git, tracked by DVC
├── notebooks/
├── src/               # reusable, tested code
├── tests/
├── .github/workflows/
├── .gitignore
├── .pre-commit-config.yaml
├── CONTRIBUTING.md
├── README.md
├── pyproject.toml + uv.lock (or requirements.txt with pinned versions)
```

2. Write a `.gitignore` that excludes datasets, checkpoints, `.env`, `__pycache__`, `.venv` and `mlruns`.
3. Import the starter code into `src/` and refactor it just enough to run from the command line, for example `python src/train.py`. Remove every hardcoded absolute path.
4. Pin the environment: `uv init + uv add scikit-learn pandas ...` to produce `uv.lock` (or `pip freeze > requirements.txt`).
5. Commit in small, well-described commits and push to `main`. This is the **only** time anyone pushes directly to `main`.
6. Create the long-lived branches and push them:

```
git checkout -b staging && git push -u origin staging
git checkout -b dev && git push -u origin dev
```

7. Turn on branch protection for `main`, `staging` and `dev`: require a pull request, at least one approval, and (after Phase 8) passing status checks. Block force pushes.

8. Add a `CONTRIBUTING.md` that states the branch naming rules, the commit message convention and your squash-vs-rebase decision.

Checkpoint: three protected branches exist; `git log` on `main` shows the initial import.

Phase 3 · Guard rails: pre-commit and secrets

On a `feat/pre-commit` branch, add a `.pre-commit-config.yaml` with at least: `ruff` (lint + format), `nbstripout`, `check-added-large-files` (limit 1 MB) and a secret scanner such as `detect-secrets` or `gitleaks`. Every member runs `pre-commit install`. Open a PR into `dev`; a teammate reviews and merges it.

Checkpoint: committing a 5 MB file or a fake API key is blocked. Take a screenshot for the report.

Phase 4 · Version the data with DVC

On a `data/initial-dataset` branch (Data owner):

```
uv add dvc          # add the extra for your remote, e.g. dvc[s3] or dvc[gdrive]
dvc init
dvc add data/raw/<dataset>.csv
dvc remote add -d storage <your-remote> # see Rules and tips
git add data/raw/<dataset>.csv.dvc data/.gitignore .dvc/config
git commit -m "data: track raw dataset with DVC"
dvc push            # ALWAYS before git push
git push -u origin data/initial-dataset
```

Open a PR into `dev`. The reviewer must clone the branch fresh, run `dvc pull`, and confirm the data arrives.

Checkpoint: the CSV is not in Git history; only its `.dvc` pointer is.

Phase 5 · Notebooks done right

On a `feat/eda-notebook` branch:

1. Create `notebooks/01-eda.ipynb` exploring the dataset.
2. Pair it with a script: `jupyter --set-formats ipynb,py:percent notebooks/01-eda.ipynb` and commit both files.
3. Move one reusable function (for example, a cleaning or feature function) into `src/` with a unit test in `tests/`, then import it back into the notebook.
4. Restart the kernel and run all cells before opening the PR.

Checkpoint: the PR diff shows no cell outputs or execution counts.

Phase 6 · A reproducible pipeline

On a `feat/dvc-pipeline` branch (Model owner):

1. Move every hyperparameter, split ratio and seed into `params.yaml`, for example:

```
seed: 42
split:
  test_size: 0.2
```

```
train:
```

```
  model: random_forest
  n_estimators: 100
  max_depth: 6
```

2. Split the code into prepare, train and evaluate stages and define them in `dvc.yaml`. `evaluate` writes `metrics.json`.
3. Set seeds everywhere randomness occurs: splitting, shuffling, model initialisation, sampling.
4. Fit preprocessing (scalers, encoders, imputers) on the training split only.
5. Log the current commit SHA with every run (MLflow, W&B, or at minimum inside `metrics.json`).
6. Run `dvc repro`, then commit `dvc.yaml`, `dvc.lock`, `params.yaml` and `metrics.json`. Run `dvc push`, then `git push`.

Checkpoint: a teammate on a fresh clone runs `dvc pull && dvc repro` and gets identical metrics.

Phase 7 · Experiments and pull requests

This phase is where most of the collaboration happens.

1. **Experiments.** Each member creates their own branch from dev, such as `exp/ali-max-depth`, and runs at least **three** experiments, for example `dvc exp run --set-param train.max_depth=10`. Compare them with `dvc exp show` and paste the table into the PR description or report. Never run experiments on uncommitted code.
2. **Promote the winner.** Apply the best experiment (`dvc exp apply <name>`), put the change on a feat/ branch, and open a PR into dev with the metrics before and after.
3. **Review each other.** Every PR is assigned to a teammate as reviewer. The reviewer fills in the review checklist (below) as a PR comment and requests changes at least once during the project. Each member must author at least **2 merged PRs** and review at least **2**.
4. **Data update.** The Data owner opens a `data/<change>` PR that modifies the dataset (remove duplicates, fix labels, add rows, or change the split). Show `git checkout + dvc checkout` moving between the old and new data version.
5. **Resolve a real conflict.** Two members each change the same line of `params.yaml` on separate branches. Merge the first PR, then the second author rebases on dev, resolves the conflict, and documents the resolution in the PR.
6. **Experiment drift.** Keep at least one `exp/` branch that is never merged, and explain in the report why it was abandoned.

Add this template as `.github/pull_request_template.md` so every PR shows the checklist:

```
## What changed and why
```

```
## Metrics (before → after)
```

```
## Review checklist
```

- [] No data leakage (no target or future information in features)
- [] Splits are fixed; preprocessing fit on training data only
- [] No hardcoded paths; runs on a teammate's machine
- [] Seeds set for shuffling, initialisation and sampling
- [] Metric computed the way the team reports it
- [] `dvc push` done before `git push` (if data or models changed)

- [] Notebook restarted and run top to bottom (if notebooks changed)
- [] Style and naming (linter passes)

Checkpoint: the PR list shows every member as both author and reviewer, with at least one "changes requested" review.

Phase 8 · CI on every pull request

On a feat/ci branch (Platform owner), add a GitHub Actions workflow in `.github/workflows/ci.yml` that runs on every PR into dev, staging and main:

1. **Lint:** `ruff check` and `ruff format --check`
2. **Unit tests:** `pytest tests/`
3. **Data checks:** schema, value ranges and null counts on the dataset or a committed sample
4. **Smoke train:** train on a small sample (a few hundred rows) to prove the pipeline runs end to end
5. **Bonus:** use [CML](#) to post the metrics table as a comment on the PR

After it merges, make these checks required in the branch protection rules.

Checkpoint: a deliberately broken test causes a red check that blocks merging.

Phase 9 · Release: dev → staging → main

1. Open a release PR from dev into staging titled `release: v1.0` that summarises the included PRs and the final metrics.
2. **Reproducibility test.** A member who did not train the final model clones the repo into a new folder and runs:

```
git clone <repo> && cd <repo> && git checkout staging
uv sync
dvc pull
dvc repro
```

They post the resulting `metrics.json` in the PR. The metrics must match the reported ones exactly.

3. Merge into staging, then open a PR from staging into main. After approval and merge, tag the release:

```
git checkout main && git pull
git tag -a model-v1.0 -m "First production model"
git push origin model-v1.0
```

4. **Optional hotfix.** Find a small bug on main, fix it on a `fix/` branch, merge it into main, tag `model-v1.0.1`, and merge main back into dev.
5. **Retrospective.** Meet as a team: what broke, what you would standardise, and what you added to `CONTRIBUTING.md` as a result.

Checkpoint: `model-v1.0` exists on main and the reproduction in step 2 succeeded.

Submission and grading

Submit the repository link. Grading is done from the repository alone, so everything below must be visible there.

REPORT.md must contain

- Team members, roles, dataset and starter-code source (with link)
- A reproducibility table for the released model: commit SHA, `params.yaml` values, data `.dvc` hash, lock file, seed, and final metrics
- The `dvc exp show` comparison of all experiments and why the winner was chosen
- Links to: the data-update PR, the conflict-resolution PR, one "changes requested" review, the release PRs, and the abandoned `exp/` branch
- Screenshots: blocked large file or secret, a failing CI check, a passing CI check
- A short retrospective: what broke, and what you added to `CONTRIBUTING.md` because of it
- One paragraph per member describing their own contribution

Rubric (100 points)

Area	What is checked	Points
Repo structure and hygiene	Standard layout, <code>.gitignore</code> , pre-commit hooks, no data or secrets in Git history, clear commits	10
Branching and protection	<code>dev / staging / main</code> protected; correct branch names; no direct pushes after Phase 2	15
Pull requests and review	Each member ≥ 2 authored and ≥ 2 reviewed PRs; checklist used; at least one "changes requested"; conflict resolved	20
Data and model versioning	DVC tracking and remote work; data-update PR; old version recoverable	15
Notebooks	Outputs stripped, jupyter text pairing, logic promoted to tested <code>src/</code>	5
Reproducible experiments	<code>params.yaml</code> , <code>dvc.yaml</code> , seeds, commit SHA logged, ≥ 3 experiments per member	15
CI	Lint, tests, data checks and smoke train run on PRs and are required	10
Release and report	Tagged <code>model-v1.0</code> , independent reproduction matches, complete <code>REPORT.md</code>	10

Area	What is checked	Points
Total		100
Bonus	CML metrics comment on PRs, or a completed hotfix with <code>model-v1.0.1</code>	+5

Individual marks may be adjusted if the Git history shows a member did not author or review their share of the work.

Datasets, starter code and tips

Pick a small tabular dataset (under about 50 MB) so pipelines and CI stay fast. Starter code can be your own earlier coursework, a scikit-learn example, or a public Kaggle notebook; credit the source in `REPORT.md`.

Dataset	Task	Source
Titanic	Binary classification	Kaggle
Wine Quality	Regression or classification	UCI ML Repository
Adult Income	Binary classification	UCI ML Repository
Heart Disease	Binary classification	UCI ML Repository
Telco Customer Churn	Binary classification	Kaggle
California Housing	Regression	<code>sklearn.datasets.fetch_california_housing</code>

Each team should use a different dataset. Tell the instructor your choice by the end of Phase 1.

Choosing a DVC remote. Teammates must be able to `dvc pull`, so the remote has to be shared. Good free options are a DagsHub repository (it provides a DVC remote), a shared Google Drive folder (`dvc[gdrive]`), or an S3-compatible bucket if your institution provides one. Never commit remote credentials: keep them in `.dvc/config.local` or environment variables.

Common mistakes that cost marks

- Committing the dataset or a model file to Git. If it happens, remove it from history with `git filter-repo`, not just a new commit.
- Running `git push` without `dvc push`, which leaves teammates with broken pointers.
- Running experiments on uncommitted changes, so the logged SHA does not match the code.
- Long-lived `exp/` branches that drift from `dev`. Rebase often or cherry-pick only what matters.
- Notebooks that only work in one cell order on one laptop.
- Approving a PR without actually running it. Reviewers should check out the branch at least once per PR that changes the pipeline.

Useful commands

